ITA06 MACHINE LEARNING

ASSIGNMENT 1

Design, Analyse and Validate an Intelligent Early-Warning Decision-Support System for Academic Risk Identification Using Concept Learning and Decision Tree Learning

Submitted By:
Name: M.Pranay
Reg No: 192525382

Task 1 – Identify and Formulate the ML Problem

1.1 Formal learning problem (T, P, E)

Element	Statement for the early-warning system
Task (T)	Given the six (Dataset A) or four (Dataset B) categorical attributes recorded after week 6, classify a student as AtRisk = Yes or No, i.e. learn $c : X \rightarrow \{0,1\}$.
Performance (P)	Primary: recall (sensitivity) on AtRisk = Yes, because a missed at-risk student is the costly error. Secondary: precision, F1 and accuracy on held-out students (Test Set T and cross-validation).
Experience (E)	Labelled history of past students: Dataset A (5 examples, 6 attributes), Dataset B (14 examples, 4 attributes) and held-out Test Set T (6 examples). Labels are assigned by the academic council at the end of the semester.

1.2 Attribute table and relevance

Attribute	Type	Values	Role	Why plausibly relevant
Attendance	Ordinal categorical	Low, Medium, High	Predictor (A, B)	Missed classes mean missed content; attendance is one of the earliest observable engagement signals.
Assignments	Binary categorical	Irregular, Regular	Predictor (A, B)	Late or missing submissions lose internal marks and signal poor time management.
StudyHours	Binary categorical	Low, High	Predictor (A only)	Time on task drives learning, but it is self-reported.
PriorCGPA	Binary categorical	Weak, Strong	Predictor (A, B)	Past performance is usually the strongest predictor of future performance (confirmed as root split in Task 6).
LMS	Binary categorical	Passive, Active	Predictor (A, B)	Online activity shows whether the student is engaging with course material.
Mentoring	Binary categorical	No, Yes	Predictor (A only)	Mentoring may reduce risk, but see the risky-attribute discussion below.
AtRisk	Binary categorical	Yes, No	Target	Council decision at the end of the semester.

Risky predictor: Mentoring. Mentoring is an intervention that is applied *because* a student is believed to be struggling, so enrolment is correlated with the outcome through selection, not only through cause. In Dataset A, E3 (Mentoring = Yes) is safe while E4 (Mentoring = Yes) is at risk, so the attribute gives no consistent signal; it can also leak the advisor's earlier judgement into the model and its effect (mentoring lowers risk) is confounded with the reason for enrolment. StudyHours is a secondary concern because it is self-reported and therefore noisy.

1.3 Target concept and instance space

The target concept is the Boolean function $c : X \rightarrow \{0,1\}$ with $c(x) = 1$ exactly when the council labels the student AtRisk = Yes, and $c(x) = 0$ otherwise.

Dataset	Cardinality calculation	Instance space $\|X\|$	Distinct training examples	Coverage
A (6 attributes)	$3 \times 2 \times 2 \times 2 \times 2 \times 2$	96	5	5/96 = 5.21%
B (4 attributes)	$3 \times 2 \times 2 \times 2$	24	14 (all 14 combinations are distinct)	14/24 = 58.33%
B + T (for reference)	same space as B	24	20	20/24 = 83.33%

Dataset A covers only about 5% of its space, so almost every future student is an unseen combination and the learner must generalise through its bias. Dataset B covers 58% of its space, but the six test students occupy six different cells that are all absent from B, so Test Set T measures extrapolation, not memory.

1.4 Learning paradigm

The paradigm is **supervised learning (classification)**. Every record carries a target label (AtRisk) supplied by the council, the output is a discrete class, and performance can be measured against known answers on T. Unlabelled data or a reward signal are not what the data provides.

1.5 Machine-learning issues

Issue	Link to these datasets / application	Mitigation
Small data	Dataset A has 5 examples for 96 cells; Dataset B has 14 examples for 24 cells; T has only 6 records.	Prefer simple models, pruning, cross-validation; collect more records (about 2,000 become available in Task 11).
Label noise / inconsistent labelling	Labels come from the council and, later, several advisors; an audit estimates 5–10% inconsistent (Task 11). E6 contradicts E1 exactly.	Noise-tolerant stopping and pruning, audit a sample of labels, use majority label per cell.
Sampling bias / coverage	B covers 58% of the instance space; T consists entirely of cells not present in B, and Attendance = Very Low appears later.	Monitor the share of unseen combinations, retrain weekly, define an explicit unseen-value policy.
Generalisation / overfitting	The fully grown tree fits 14/14 training rows but 3/6 test rows (Task 8); four of its eight leaves hold a single student.	Depth limit, minimum-gain threshold, leave-one-out evaluation.
Fairness and privacy	PriorCGPA and LMS activity can act as proxies for socio-economic background; flagging labels students and exposes their LMS logs.	Audit recall and false-positive rate by student group, restrict access, show reasons and allow appeal.
Label delay and drift	Attributes are taken at week 6 but the label is only known at semester end; new online study mode changes attribute meaning.	Retrain every semester or weekly with new records; track attribute-value distribution drift.

1.6 Cost of errors and performance measure

A **false negative** (an at-risk student not flagged) is costlier: the student receives no counselling or remedial help and may fail, which is the harm the system exists to prevent. A **false positive** costs one counselling slot and some stigma, which is real but recoverable. I therefore emphasise **recall on AtRisk = Yes**, report precision and F1 next to it so that the flag rate stays manageable, and use accuracy only as supporting evidence. The council's later 90% recall requirement (Task 11) confirms this choice.

Task 2 – Analyse the Learning Paradigm

2.1 Comparison matrix

Criterion	Supervised	Unsupervised	Reinforcement
Availability of labels	Used directly: AtRisk is known for past students.	Labels ignored; the 5/14 labelled rows add nothing to the grouping.	No labels; only a reward after actions.
Form of output	Class (Yes/No), optionally with probability.	Cluster id or profile.	A policy: which intervention to take in which state.
Feedback signal	Error against true label.	None (internal cohesion measures).	Delayed scalar reward (pass/fail minus intervention cost).
Data needed	Tens to thousands of labelled students.	Many unlabelled students.	Very many student-semester trajectories; one reward per semester.
How success is evaluated	Recall, precision, F1 on held-out students.	Silhouette or interpretability; cannot measure missed at-risk students.	Cumulative reward; needs off-policy evaluation, because experimenting on students is limited.
Interpretability for advisors	High with trees and rules (IF ... THEN).	Medium: cluster descriptions, but not a reason for a flag.	Low: policy values are hard to justify to a student.
Main risk	Label noise and bias in historical labels.	Clusters that do not align with failure.	Unsafe exploration and confounded rewards.

2.2 Unsupervised learning in this scenario

Concrete use: cluster the week-6 records with k-modes into behavioural profiles (for example “disengaged”: Low attendance, Irregular, Passive; “capable but irregular”: Strong CGPA, Irregular), so advisors can design a different support programme for each profile. **What it cannot deliver:** a decision. Clusters are built without the AtRisk label, so there is no guarantee that a cluster boundary coincides with failing; assigning a cluster to “at risk” needs labels again; the number of missed at-risk students cannot be measured; and the output gives no per-student reason connected to the outcome. For example, in Dataset B the students in the group (High attendance, Irregular, Passive, Weak) are at risk (row 9) while (High, Irregular, Active, Strong) are safe (row 10), and such groups could be merged by a distance measure.

2.3 Reinforcement learning in a follow-up problem

Follow-up problem: choosing which intervention to schedule for a flagged student. **State:** week number, the six attributes, interventions already given and the student's current engagement. **Action:** {no action, email nudge, advisor meeting, remedial class, mentoring assignment}. **Reward:** +1 if the student passes, minus a cost for each intervention used (staff time), and a large penalty if the student fails.

RL is unsuitable for the *initial* classification because (i) there is no sequential decision: one prediction is made at week 6; (ii) correct labels already exist, so the learner does not have to discover them by trial; (iii) feedback arrives once per student per semester, far too sparse for RL; and (iv) exploring by deliberately not helping students is ethically unacceptable.

2.4 Selected paradigm, counter-argument and rebuttal

Selection: supervised learning, because labels exist, the decision is a one-step classification, and recall can be measured and controlled. **Strongest argument against:** the labels are few and noisy (14 rows, later 5–10% inconsistent), so a supervised model may learn the advisors' inconsistencies or overfit, while unsupervised methods do not depend on labels. **Rebuttal:** unsupervised learning cannot produce the required decision or explanation at all, so avoiding labels is not an option; the label problem is handled inside supervised learning with pruning, minimum-gain thresholds, cross-validation and a label audit (Tasks 8 and 11). Task 8 already shows the failure mode (a full tree reaching 3/6 on T) and its control (a depth-1 tree reaching 5/6).

2.5 Hybrid proposal

Clustering + supervised tree (semi-supervised flavour). Cluster all current-semester students (labelled or not) into profiles, use the profile as an extra input, and train the interpretable tree on the labelled history. Benefit: students

with attribute combinations never seen in the labelled data (such as Very Low attendance) are placed in a profile with similar students, which gives the tree a sensible route and improves coverage, while the final flag stays supervised and explainable. A later hybrid is supervised risk prediction feeding an RL intervention planner.

Task 3 – Formulate the Concept Learning Task

3.1 Target concept and labelled examples

c : X → {0,1}, c(x) = 1 iff AtRisk = Yes. Instance space size is 96.

Example	⟨Attendance, Assignments, StudyHours, PriorCGPA, LMS, Mentoring⟩	AtRisk	c(x)	Type
E1	<Low, Irregular, Low, Weak, Passive, No>	Yes	1	Positive
E2	<Low, Irregular, Low, Strong, Passive, No>	Yes	1	Positive
E3	<High, Regular, High, Strong, Active, Yes>	No	0	Negative
E4	<Low, Irregular, High, Weak, Passive, Yes>	Yes	1	Positive
E5	<Medium, Irregular, High, Strong, Active, No>	No	0	Negative

3.2 Hypothesis representation

A hypothesis is a conjunction of six constraints, each a specific value, "?" (any value) or "Ø" (no value). Most general hypothesis: ⟨?, ?, ?, ?, ?, ?⟩ (classifies every instance positive). Most specific hypothesis: ⟨Ø, Ø, Ø, Ø, Ø, Ø⟩ (classifies every instance negative). E1 as a hypothesis: ⟨Low, Irregular, Low, Weak, Passive, No⟩. E3 as a hypothesis: ⟨High, Regular, High, Strong, Active, Yes⟩.

3.3 Size of the spaces

Quantity	Calculation	Result
Instance space	3 × 2 × 2 × 2 × 2 × 2	96
Syntactically distinct hypotheses	Attendance: 3 values + ? + Ø = 5; each other attribute: 2 values + ? + Ø = 4. $5 \times 4^5 = 5 \times 1024$	5,120
Semantically distinct hypotheses	Allow ? but not Ø per attribute: $(3+1) \times (2+1)^5 = 4 \times 243 = 972$, plus 1 for the single "classify nothing" hypothesis (every hypothesis containing Ø is equivalent)	973

3.4 Three hypotheses and their ordering

Hypothesis	Definition	Justification
h1	⟨Low, ?, ?, ?, Passive, ?⟩	Covers every instance with Attendance = Low and LMS = Passive.
h2	⟨Low, Irregular, ?, ?, Passive, ?⟩	Has all constraints of h1 plus Assignments = Irregular. Every instance covered by h2 is covered by h1, but not conversely (⟨Low, Regular, ..., Passive, ...⟩ satisfies h1 only), so h1 ≥g h2 (strictly).
h3	⟨Medium, ?, ?, ?, ?, ?⟩	Incomparable with both. ⟨Medium, Regular, Low, Weak, Active, No⟩ is covered by h3 but by neither h1 nor h2, so h3 is not ≤ them; ⟨Low, Irregular, Low, Weak, Passive, No⟩ is covered by h1 and h2 but not h3, so h3 is not ≥ them.

3.5 Consistency with Dataset A

Example	Label	h_a = ⟨Low,?,?,?,Passive,?⟩	h_b = ⟨?,Irregular,?,?,?,?⟩	h_c = ⟨Low,Irregular,Low,?,?,?⟩
E1	+	covers ✓	covers ✓	covers ✓
E2	+	covers ✓	covers ✓	covers ✓
E3	−	no ✓	no ✓	no ✓
E4	+	covers ✓	covers ✓	no ✗
E5	−	no ✓	covers ✗	no ✓
Consistent?		Yes	No – E5	No – E4

h_a is consistent: it covers E1, E2, E4 (positive) and rejects E3, E5 (negative). h_b is inconsistent because it covers E5 (Irregular) although E5 is negative. h_c is inconsistent because it does not cover E4 (StudyHours = High) although E4 is positive.

3.6 Satisfying an instance versus being consistent

A hypothesis *satisfies* an instance when h(x) = 1; this concerns one instance. It is *consistent* with the training data when h(x) = c(x) for **every** example, i.e. it satisfies all positives and none of the negatives. h_b satisfies E1, E2, E4 and also E5, but because it satisfies the negative E5 it is not consistent; h_c satisfies E1 and E2 but fails to satisfy the positive E4, so it is not consistent either; only h_a is both.

Task 4 – Apply the Candidate Elimination Algorithm

Initialisation: $S_0 = \{\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle\}$, $G_0 = \{\langle ?, ?, ?, ?, ?, ? \rangle\}$. Examples are processed in the order E1 → E5.

Step	Ex.	Label	Action taken	S	G	Reason
0	–	–	Initialise	{⟨Ø,Ø,Ø,Ø,Ø,Ø⟩}	{⟨?,?,?,?,?,?⟩}	Most specific / most general boundaries.
1	E1	+	Positive: S_0 does not cover E1; minimal generalisation of S gives E1 itself. G_0 covers E1, so nothing is removed.	{<Low, Irregular, Low, Weak, Passive, No>}	{<?, ?, ?, ?, ?, ?>}	First positive example anchors S.
2	E2	+	Positive: S_1 does not cover E2. Only PriorCGPA differs (Weak vs Strong), so that attribute is minimally generalised to ?. G_2 is unchanged because ⟨?,?,?,?,?,?⟩ covers E2.	{<Low, Irregular, Low, ?, Passive, No>}	{<?, ?, ?, ?, ?, ?>}	Positive examples only generalise S.
3	E3	−	Negative: S_2 does not cover E3, so S is unchanged. G_2 covers E3, so it is replaced by its minimal specialisations. Seven candidates are generated (one attribute set to a value different from E3's): Attendance=Low, Medium; Assignments=Irregular; StudyHours=Low; PriorCGPA=Weak; LMS=Passive; Mentoring=No. Rejected as not more general than S_2: ⟨Medium,…⟩ and ⟨…,Weak,…⟩ (S_2 has Attendance=Low and PriorCGPA=?). Five are kept; none is more specific than another.	{<Low, Irregular, Low, ?, Passive, No>}	{<Low, ?, ?, ?, ?, ?>; <?, Irregular, ?, ?, ?, ?>; <?, ?, Low, ?, ?, ?>; <?, ?, ?, ?, Passive, ?>; <?, ?, ?, ?, ?, No>}	Negative examples only specialise G; candidates must stay ≥ S.
4	E4	+	Positive: first remove from G members that do not cover E4: ⟨?,?,Low,?,?,?⟩ (E4 has StudyHours = High) and ⟨?,?,?,?,?,No⟩ (E4 has Mentoring = Yes). S_3 does not cover E4: StudyHours Low→? and Mentoring No→?.	{<Low, Irregular, ?, ?, Passive, ?>}	{<Low, ?, ?, ?, ?, ?>; <?, Irregular, ?, ?, ?, ?>; <?, ?, ?, ?, Passive, ?>}	G members must cover every positive example.
5	E5	−	Negative: S_4 does not cover E5 (Attendance Medium), so S is unchanged. Only ⟨?,Irregular,?,?,?,?⟩ covers E5, so it is specialised. Of the seven candidates, ⟨Low,Irregular,…⟩ and ⟨?,Irregular,…,Passive,?⟩ are kept; ⟨High,Irregular,…⟩, ⟨?,Irregular,Low,…⟩, ⟨?,Irregular,…,Weak,…⟩ and ⟨?,Irregular,…,Yes⟩ are rejected (not more general than S_4). The two kept ones are then removed because each is less general than another member of G (⟨Low,?,…⟩ and ⟨?,…,Passive,?⟩).	{<Low, Irregular, ?, ?, Passive, ?>}	{<Low, ?, ?, ?, ?, ?>; <?, ?, ?, ?, Passive, ?>}	Specialisations that are subsumed by another G member are redundant.

Final boundaries: $S_5 = \{\langle Low, Irregular, ?, ?, Passive, ? \rangle\}$; $G_5 = \{\langle Low, ?, ?, ?, ?, ? \rangle, \langle ?, ?, ?, ?, Passive, ? \rangle\}$.

Task 5 – Analyse Version Space and Inductive Bias

5.1 Version space

Name	Hypothesis	Boundary
h1	⟨Low, Irregular, ?, ?, Passive, ?⟩	S_5
h2	⟨Low, Irregular, ?, ?, ?, ?⟩	
h3	⟨Low, ?, ?, ?, Passive, ?⟩	
h4	⟨Low, ?, ?, ?, ?, ?⟩	G_5
h5	⟨?, Irregular, ?, ?, Passive, ?⟩	
h6	⟨?, ?, ?, ?, Passive, ?⟩	G_5

The version space contains **6 hypotheses**: the three constraints that remain in S_5 are Attendance = Low, Assignments = Irregular and LMS = Passive, and a member must keep Attendance = Low or LMS = Passive (to stay below a G member), giving $2^3 - 2 = 6$ subsets (the empty set and Assignments alone are excluded).

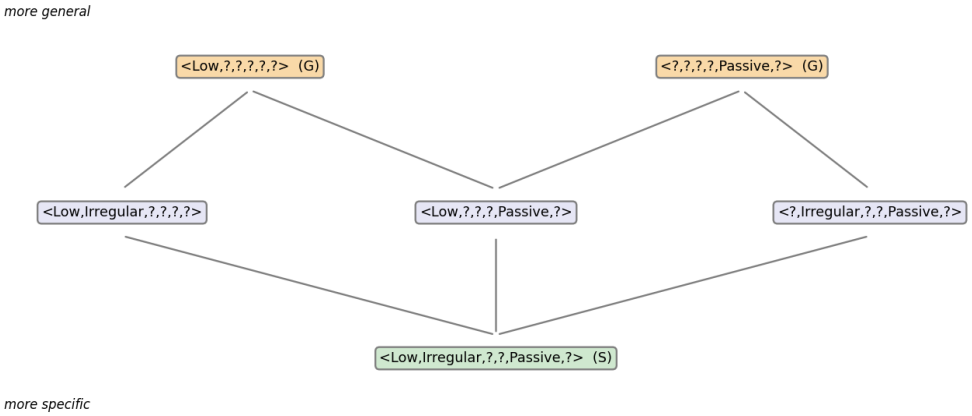


Figure 1 – General-to-specific lattice of the version space (G at the top, S at the bottom).

Why ⟨?, Irregular, ?, ?, ?, ?⟩ is not in the version space: it covers E5 = ⟨Medium, Irregular, High, Strong, Active, No⟩, which is a negative example, so it is inconsistent. It is also not more specific than any member of G_5 (it neither requires Low attendance nor Passive LMS).

5.2 Classification by voting

Hypothesis	Q1 = ⟨Low, Regular, High, Weak, Passive, No⟩	Q2 = ⟨Low, Irregular, Low, Strong, Passive, Yes⟩
h1 ⟨Low,Irr,?,?,Pas,?⟩	− (Regular)	+
h2 ⟨Low,Irr,?,?,?,?⟩	− (Regular)	+
h3 ⟨Low,?,?,?,Pas,?⟩	+	+
h4 ⟨Low,?,?,?,?,?⟩	+	+
h5 ⟨?,Irr,?,?,Pas,?⟩	− (Regular)	+
h6 ⟨?,?,?,?,Pas,?⟩	+	+
Vote	3 positive : 3 negative	6 positive : 0 negative

Q2 is classified unanimously as positive (AtRisk = Yes); Q1 is **ambiguous**, with a 3–3 split, because the hypotheses disagree about whether Assignments = Irregular is required. **Most informative next query:** an instance that splits the six hypotheses as evenly as possible, such as Q1 itself (or any Low-attendance, Regular, Passive student). Whatever its label, it removes exactly half of the version space: if Yes, h1, h2, h5 go; if No, h3, h4, h6 go. This is a guaranteed one bit of information; Q2 would give none.

5.3 Four concepts

Concept	Meaning	Example from this problem
Training data	Labelled examples observed.	E1–E5 with labels Yes, Yes, No, Yes, No.
Hypothesis space	All hypotheses the learner may consider.	All 973 semantically distinct conjunctions over the six attributes.
Version space	Hypotheses in the hypothesis space consistent with all training data.	The 6 hypotheses h1–h6, bounded by S_5 and G_5.
Inductive bias	Assumptions that let the learner classify unseen instances.	Target concept is a conjunction of attribute constraints, and the training data are noise free.

5.4 Inductive bias and its limits

Bias of Candidate Elimination here: the target concept is assumed to be representable as a *conjunction* of attribute constraints (a restriction bias), and the data are assumed consistent. Without this bias no unseen instance could be classified.

Demonstration. Add the positive example ⟨High, Regular, High, Strong, Passive, No⟩ to E1–E5. G members that do not cover it are removed: ⟨Low,?,?,?,?,?⟩ (its Attendance is High). S_5 is generalised: Attendance Low→?, Assignments Irregular→?, giving S_6 = {⟨?,?,?,?,Passive,?⟩}. G_6 = {⟨?,?,?,?,Passive,?⟩}. The version space collapses from 6 hypotheses to **one**: "LMS = Passive ⇒ at risk". The learner has been forced to drop two attributes from a single example, and any later positive example with Active LMS (such as E8 in Task 11) will destroy the version space entirely, because a conjunction cannot express "Passive OR Low attendance".

Unbiased learner: if the hypothesis space were the power set of the instance space, there would be 2^{96} = 79,228,162,514,264,337,593,543,950,336 ≈ 7.92×10^{28} possible concepts. After 5 training examples, 2^{91} ≈ 2.48e+27 remain consistent; after 6 examples 2^{90}. S would be just the disjunction of the observed positives and G the negation of the observed negatives, so for every unseen instance exactly half of the version space predicts Yes and half predicts No. The learner can only memorise; without a bias (an assumption beyond the data) it cannot generalise.

Task 6 – Construct the Decision Tree (ID3)

6.1 Entropy of the full training set

Dataset B has 14 examples: 7 Yes and 7 No. Entropy(S) = $-(7/14)\log_2(7/14) - (7/14)\log_2(7/14) = -0.5 \times (-1) - 0.5 \times (-1) =$ **1.0000**.

6.2 Root split

Attribute	Value	n	Yes	No	Entropy	Weighted contribution	Gain
Attendance	Low	5	4	1	0.7219	5/14 × 0.7219 = 0.2578	0.1636
Attendance	Medium	5	2	3	0.9710	5/14 × 0.9710 = 0.3468	
Attendance	High	4	1	3	0.8113	4/14 × 0.8113 = 0.2318	
Assignments	Irregular	8	5	3	0.9544	8/14 × 0.9544 = 0.5454	0.0611
Assignments	Regular	6	2	4	0.9183	6/14 × 0.9183 = 0.3936	
LMS	Passive	7	5	2	0.8631	7/14 × 0.8631 = 0.4316	0.1369
LMS	Active	7	2	5	0.8631	7/14 × 0.8631 = 0.4316	
PriorCGPA	Weak	7	6	1	0.5917	7/14 × 0.5917 = 0.2958	0.4083
PriorCGPA	Strong	7	1	6	0.5917	7/14 × 0.5917 = 0.2958	

Information gain: Attendance 0.1636, Assignments 0.0611, LMS 0.1369, **PriorCGPA 0.4083**. Example calculation for PriorCGPA: Weak (7: 6 Yes, 1 No): $-(6/7)\log_2(6/7) - (1/7)\log_2(1/7) = 0.1906 + 0.4011 = 0.5917$; Strong gives the same by symmetry (1 Yes, 6 No); weighted entropy = 0.5 × 0.5917 + 0.5 × 0.5917 = 0.5917; gain = 1.0000 − 0.5917 = 0.4083. **PriorCGPA is chosen as root.**

6.3 Impure branches, stopping and tie-breaking

Stopping criteria: (1) all examples in the node have the same class (pure); (2) no attributes remain; (3) the subset is empty (use the parent's majority class); (4) in the controlled variants of Task 8, maximum depth, minimum samples to split or minimum gain. **Tie-breaking:** if two attributes have the same gain, the one that comes first in the declared order (Attendance, Assignments, LMS, PriorCGPA) is selected; if a leaf has equal Yes/No counts, it predicts **Yes** (the costlier error is a missed at-risk student).

PriorCGPA = Weak (7 examples: 6 Yes, 1 No; entropy 0.5917)

Attribute	Value	n	Yes	No	Entropy	Weighted contribution	Gain
Attendance	Low	3	3	0	0.0000	3/7 × 0.0000 = 0.0000	0.3060
Attendance	Medium	2	2	0	0.0000	2/7 × 0.0000 = 0.0000	
Attendance	High	2	1	1	1.0000	2/7 × 1.0000 = 0.2857	
Assignments	Irregular	4	4	0	0.0000	4/7 × 0.0000 = 0.0000	0.1981
Assignments	Regular	3	2	1	0.9183	3/7 × 0.9183 = 0.3936	
LMS	Passive	4	4	0	0.0000	4/7 × 0.0000 = 0.0000	0.1981
LMS	Active	3	2	1	0.9183	3/7 × 0.9183 = 0.3936	

Gains: Attendance 0.3060, Assignments 0.1981, LMS 0.1981. Attendance is selected.

PriorCGPA = Strong (7 examples: 1 Yes, 6 No; entropy 0.5917)

Attribute	Value	n	Yes	No	Entropy	Weighted contribution	Gain
Attendance	Low	2	1	1	1.0000	2/7 × 1.0000 = 0.2857	0.3060
Attendance	Medium	3	0	3	0.0000	3/7 × 0.0000 = 0.0000	
Attendance	High	2	0	2	0.0000	2/7 × 0.0000 = 0.0000	
Assignments	Irregular	4	1	3	0.8113	4/7 × 0.8113 = 0.4636	0.1281
Assignments	Regular	3	0	3	0.0000	3/7 × 0.0000 = 0.0000	

Attribute	Value	n	Yes	No	Entropy	Weighted contribution	Gain
LMS	Passive	3	1	2	0.9183	3/7 × 0.9183 = 0.3936	0.1981
LMS	Active	4	0	4	0.0000	4/7 × 0.0000 = 0.0000	

Gains: Attendance 0.3060, Assignments 0.1281, LMS 0.1981. Attendance is selected.

Node PriorCGPA = Weak, Attendance = High (2 examples: rows 9 and 12)

Attribute	Value	n	Yes	No	Entropy	Weighted contribution	Gain
Assignments	Irregular	1	1	0	0.0000	1/2 × 0.0000 = 0.0000	1.0000
Assignments	Regular	1	0	1	0.0000	1/2 × 0.0000 = 0.0000	
LMS	Passive	1	1	0	0.0000	1/2 × 0.0000 = 0.0000	1.0000
LMS	Active	1	0	1	0.0000	1/2 × 0.0000 = 0.0000	

Assignments and LMS tie at gain 1.0000 (both separate the Yes and No example perfectly). By the tie-breaking rule **Assignments** is chosen. All resulting nodes are pure, so growth stops.

Node PriorCGPA = Strong, Attendance = Low (2 examples: rows 2 and 4)

Attribute	Value	n	Yes	No	Entropy	Weighted contribution	Gain
Assignments	Irregular	1	1	0	0.0000	1/2 × 0.0000 = 0.0000	1.0000
Assignments	Regular	1	0	1	0.0000	1/2 × 0.0000 = 0.0000	
LMS	Passive	1	1	0	0.0000	1/2 × 0.0000 = 0.0000	1.0000
LMS	Active	1	0	1	0.0000	1/2 × 0.0000 = 0.0000	

Assignments and LMS tie at gain 1.0000 (both separate the Yes and No example perfectly). By the tie-breaking rule **Assignments** is chosen. All resulting nodes are pure, so growth stops.

The nodes PriorCGPA=Weak/Attendance=Low (3 Yes), =Medium (2 Yes), PriorCGPA=Strong/Attendance=Medium (3 No) and =High (2 No) are pure and become leaves.

6.4 Final tree, rules and statistics

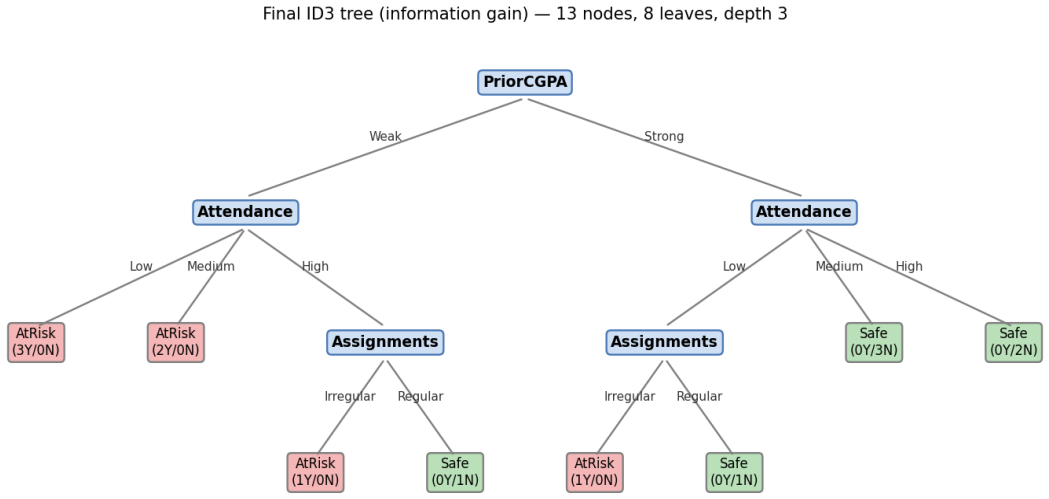


Figure 2 – Final ID3 tree built from Dataset B.

```
IF PriorCGPA = Weak AND Attendance = Low THEN AtRisk = Yes   [n=3, {'Yes': 3}]
IF PriorCGPA = Weak AND Attendance = Medium THEN AtRisk = Yes   [n=2, {'Yes': 2}]
IF PriorCGPA = Weak AND Attendance = High AND Assignments = Irregular THEN AtRisk = Yes   [n=1, {'Yes': 1}]
IF PriorCGPA = Weak AND Attendance = High AND Assignments = Regular THEN AtRisk = No   [n=1, {'No': 1}]
IF PriorCGPA = Strong AND Attendance = Low AND Assignments = Irregular THEN AtRisk = Yes   [n=1, {'Yes': 1}]
IF PriorCGPA = Strong AND Attendance = Low AND Assignments = Regular THEN AtRisk = No   [n=1, {'No': 1}]
IF PriorCGPA = Strong AND Attendance = Medium THEN AtRisk = No   [n=3, {'No': 3}]
IF PriorCGPA = Strong AND Attendance = High THEN AtRisk = No   [n=2, {'No': 2}]
```

Statistic	Value
Depth	3
Number of nodes	13 (5 internal + 8 leaves)
Number of leaves	8
Training accuracy	14/14 = 100%

Task 7 – Analyse Heuristic Search in Decision Tree Learning

7.1 Search formulation

Element	ID3 as search
Starting state	A single leaf containing all 14 examples (majority class Yes by tie rule).
Operator	Replace one impure leaf by an internal node that tests one unused attribute and creates one child per value.
Evaluation heuristic	Information gain of each candidate attribute (greedy hill climbing).
Termination test	Node pure, no attributes left, empty subset (or a pre-pruning limit).

Decision node	Candidate attributes evaluated	Gain computations
Root (14 examples)	Attendance, Assignments, LMS, PriorCGPA	4
PriorCGPA = Weak	Attendance, Assignments, LMS	3
Weak, Attendance = High	Assignments, LMS	2
PriorCGPA = Strong	Attendance, Assignments, LMS	3
Strong, Attendance = Low	Assignments, LMS	2
Total		14

Size of the searched space. The instance space of Dataset B has 24 points, so there are 2^{24} = 16,777,216 Boolean concepts and a tree can represent every one of them (it can represent any function of the four discrete attributes). Candidate Elimination in Task 3 searches only 973 semantically distinct conjunctions (for six attributes). The tree's space is far less restricted: it has almost no *restriction bias*, but ID3 expresses a *preference bias* through its search, favouring short trees with high-gain attributes near the root. ID3 evaluated only 14 gains instead of exploring a space of that size, but because its space is so large and the data so small, its preference bias, not its representation, determines what it learns.

7.2 Information gain versus gain ratio

Attribute	Information gain	Split information	Gain ratio
PriorCGPA	0.4083	1.0000	0.4083
Attendance	0.1636	1.5774	0.1037
StudentID (unique)	1.0000	3.8074	0.2626

Split information for Attendance (5 Low, 5 Medium, 4 High) = $-2 \times (5/14)\log_2(5/14) - (4/14)\log_2(4/14) = 1.5774$; for PriorCGPA (7/7) it is 1.0000. Gain ratio divides the gain by split information, which grows with the number of values and the evenness of the split, so it penalises attributes that fragment the data. Both measures choose PriorCGPA at the root.

StudentID. Each of 14 values contains one example, so every subset is pure (entropy 0). Weighted entropy = 0, so gain = 1.0000 − 0 = **1.0000**, the maximum, higher than any real attribute. It would be a very poor choice: the tree would simply memorise the training students, and a future student has an ID that does not exist in the tree, so no prediction is possible. Gain ratio reduces its score to 1.0000 / $\log_2 14$ = 0.2626, below PriorCGPA's 0.4083.

Selection adopted: information gain with identifier-like attributes excluded, because it is simple, picks the same root as gain ratio and produces the same-size tree here. Gain ratio becomes preferable when attribute cardinality grows (Task 11, where Attendance gains a fourth value).

For completeness, the gain-ratio tree has the same root, 13 nodes, 8 leaves, depth 3, training accuracy 100% and test accuracy 0.5000; it differs only in lower splits (Assignments / LMS instead of Attendance), where the choices rest on 7 examples.

7.3 Greedy versus Attendance-rooted tree

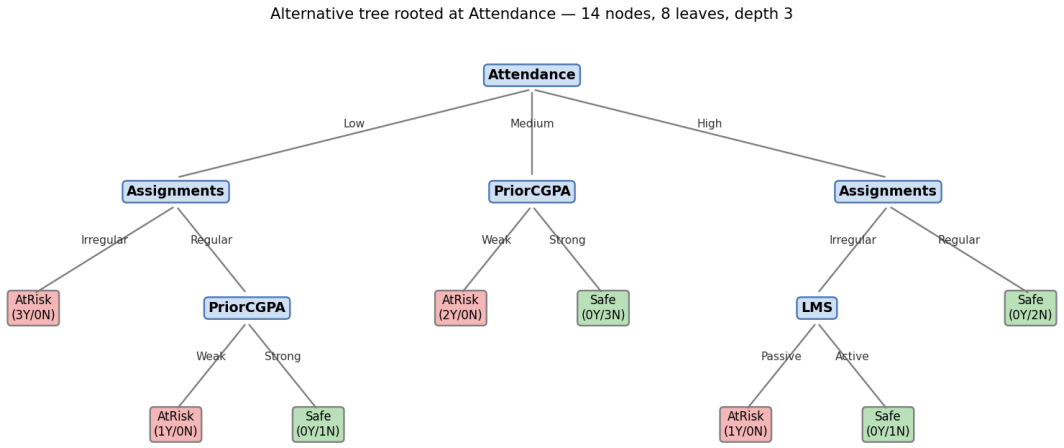


Figure 3 – Alternative tree with Attendance forced at the root.

In the Attendance = Low branch (4 Yes, 1 No; entropy 0.7219) Assignments and PriorCGPA both give weighted entropy 0.4000 (gain 0.3219), so the tie rule selects Assignments; the rest follows the same stopping rules.

Measure	Greedy tree (root PriorCGPA)	Attendance-rooted tree
Nodes	13	14
Leaves	8	8
Depth	3	3
Training accuracy	14/14 = 100%	14/14 = 100%
Test accuracy on T	3/6 = 0.5000	2/6 = 0.3333

The greedy choice produced a slightly smaller tree (13 versus 14 nodes) with the same number of leaves and depth and a better test accuracy, but the difference on six test students is not statistically meaningful. ID3 never backtracks because it keeps no record of alternative attribute choices; revisiting them would multiply the cost (one tree per alternative per node) and it commits to each split using only local information gain.

Advantages. (1) Efficiency: only 14 gain computations built the whole tree instead of searching 16.7 million concepts. (2) The heuristic finds a compact, interpretable tree: PriorCGPA's gain (0.4083) is far above the others, so a strong predictor is used first. **Limitations.** (1) Greedy choices are not globally optimal: the best root by gain does not guarantee the smallest tree (the two trees above differ only slightly and neither is provably minimal), and the depth-1 tree already scores 12/14 with just 3 nodes. (2) With very small subsets the heuristic is unreliable: two attributes tie at gain 1.0000 on two examples, and the choice between Assignments and LMS is arbitrary though it changes the rules produced.

Task 8 – Analyse Overfitting and Underfitting

8.1 Diagnosis

Model	Nodes	Leaves	Training accuracy	Test accuracy (T)	Gap (train – test)
M0 (depth 0)	1	1	7/14 = 0.5000	3/6 = 0.5000	+0.0000
M1 (depth 1)	3	2	12/14 = 0.8571	5/6 = 0.8333	+0.0238
M2 (full tree)	13	8	14/14 = 1.0000	3/6 = 0.5000	+0.5000

Test example	True	M0	M1	M2
T1	No	Yes	No	Yes
T2	No	Yes	Yes	Yes
T3	Yes	Yes	Yes	No
T4	Yes	Yes	Yes	Yes
T5	No	Yes	No	No
T6	Yes	Yes	Yes	Yes

M0 predicts Yes for everyone (tie rule): training and test accuracy are both 0.5000, with 1 node. It **underfits** (high bias: both errors are high and similar). M2 reaches 100% training accuracy but only 0.5000 on T: a gap of 0.5000 with 13 nodes and 8 leaves, so it **overfits**. **Best generalisation is at M1**, depth 1, with 0.8571 training and 0.8333 test accuracy and a small gap (0.0238).

8.2 Causes

(i) Only 14 training students and 6 test students, so each split is decided by a handful of records; (ii) the training data cover 14 of 24 cells, and every test student is in a cell the tree never saw, so the test is extrapolation into unseen combinations; (iii) four of the eight leaves of M2 hold a single student and two more hold two students, so they encode individual students rather than rules, such as Strong/Low/Irregular → Yes from the single row 2; (iv) possible label noise: if one of those single-student leaves is mislabelled, the full tree reproduces the error. T3 (High attendance, Regular, Passive, Weak → Yes) and T1 (Low, Irregular, Active, Strong → No) show that the depth-3 interactions in the tree are not stable.

8.3 Control methods

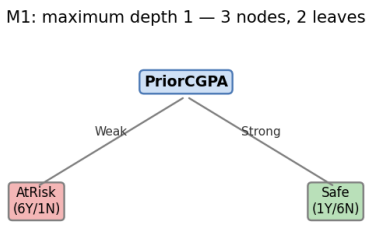


Figure 4 – Maximum depth 1 (identical to M1).

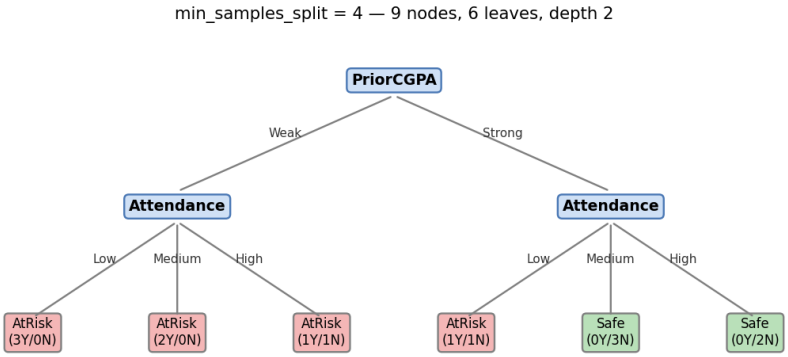


Figure 5 – min_samples_split = 4 (nodes of size 2 are not split; ties predict Yes).

Method	Nodes	Leaves	Train accuracy	Test accuracy	Comment
Max depth = 1	3	2	12/14 = 0.8571	5/6 = 0.8333	
Min information gain = 0.35	3	2	12/14 = 0.8571	5/6 = 0.8333	
min_samples_split = 4	9	6	12/14 = 0.8571	3/6 = 0.5000	
LOO cross-validation (selection)	–	–	–	–	Depth 0: 0.00 (artefact), depth 1: 0.85, depth 2: 0.85, depth 3: 0.55 → choose depth 1

Tied-leaf rule: when a leaf has equal Yes and No counts it predicts Yes (the costlier error is a missed at-risk student). Applying the rule: leaves (Weak, High) and (Strong, Low) in the min_samples_split = 4 tree each have 1 Yes and 1 No and both predict Yes. Maximum depth 1 and a minimum-gain threshold of 0.35 produce exactly the same tree: the best gain at the depth-1 nodes is 0.3060, below the threshold, so growth stops. min_samples_split = 4 improves training behaviour (12/14) but not test accuracy (3/6): it removes the single-student splits but keeps the Attendance split, which is itself unstable on this data. With the opposite tie rule (tie → No) it also scores 3/6, so the result is not an artefact of the tie rule. A *reduced-error post-pruning* with a validation set was not used, because there are not enough students to hold out a validation set without destroying the training set.

8.4 Final complexity and reliability

Final choice: maximum depth 1 (equivalently minimum gain 0.35): it has the best test accuracy (0.8333), the best-or-tied LOO accuracy (0.85, tied with depth 2) and the smallest tree among those, so by parsimony it is preferred. **Reliability:** six test students make each student worth 16.7 percentage points; the 95% Wilson interval for 5/6 is roughly 44% to 97%, so the conclusion “depth 1 beats the full tree” is suggestive, not proven. Improvement: use repeated stratified k-fold cross-validation on all labelled records (now about 2,000), report confidence intervals, and use a test set that mixes seen and unseen cells.

Task 9 – Implement and Test the ML Techniques

9.1 Implementation summary

Component	Implementation (ml_core.py)
Concept-learning representation	Tuples with "?" and "Ø"; covers(h, x), more_general_or_equal(h1, h2), min_generalize, min_specializations.
Candidate Elimination	candidate_elimination(): S and G are Python sets; prints S and G after every example, records a trace, and returns the example at which the version space becomes empty.
Decision tree	entropy, info_gain, split_info, recursive id3() with pure-node, no-attribute, max_depth, min_samples_split, min_gain, min_leaf and forced_root controls; predict(), rules(), show(), stats(); deterministic tie-breaking.
Reproducibility	Random seed 42 everywhere (random.seed / random.Random(seed)); no external ML library was used for either algorithm.

9.2 README and run instructions

```
README – ITA06 Assignment 1
Files: ml_core.py (algorithms), gen.py (Dataset C generator), tests.py (TC1-TC8), experiments.py (Task 10/11),
     run_main.py (prints CE trace and ID3 trees), figs.py (figures).
Requirements: Python 3.9+, matplotlib (plots only).  No scikit-learn is needed.
Run:  python run_main.py    # S/G after every example, entropy/gain per node, tree, rules, predictions on T
     python tests.py       # test-case table (all deterministic)
     python experiments.py  # Task 10 and Task 11 results, results.json and figures
Dataset C: 1000 records, seed 42, rule score = 2[Att=Low] + [Att=Medium] + [Assign=Irregular] + [LMS=Passive] + [CGPA=Weak];
        AtRisk = Yes if score >= 3; label-noise rate is a parameter of gen(n, seed, noise).
```

9.3 Console log: Candidate Elimination on Dataset A (condensed)

```
S0 = {<Ø, Ø, Ø, Ø, Ø, Ø>}
G0 = {<?, ?, ?, ?, ?, ?>}

Step 1: E1 (Low, Irregular, Low, Weak, Passive, No) label=Yes
   - S generalised <Ø, Ø, Ø, Ø, Ø, Ø> -> <Low, Irregular, Low, Weak, Passive, No>
S1 = {<Low, Irregular, Low, Weak, Passive, No>}
G1 = {<?, ?, ?, ?, ?, ?>}

Step 2: E2 (Low, Irregular, Low, Strong, Passive, No) label=Yes
   - S generalised <Low, Irregular, Low, Weak, Passive, No> -> <Low, Irregular, Low, ?, Passive, No>
S2 = {<Low, Irregular, Low, ?, Passive, No>}
G2 = {<?, ?, ?, ?, ?, ?>}

Step 3: E3 (High, Regular, High, Strong, Active, Yes) label=No
   - kept specialisation <Low, ?, ?, ?, ?, ?>
   - rejected <Medium, ?, ?, ?, ?, ?> (not more general than any s in S)
   - kept specialisation <?, Irregular, ?, ?, ?, ?>
   - kept specialisation <?, ?, Low, ?, ?, ?>
   - rejected <?, ?, ?, Weak, ?, ?> (not more general than any s in S)
   - kept specialisation <?, ?, ?, ?, Passive, ?>
   - kept specialisation <?, ?, ?, ?, ?, No>
S3 = {<Low, Irregular, Low, ?, Passive, No>}
G3 = {<?, ?, ?, ?, ?, No>; <?, ?, ?, ?, Passive, ?>; <?, ?, Low, ?, ?, ?>; <?, Irregular, ?, ?, ?, ?>; <Low, ?, ?, ?, ?, ?>}

Step 4: E4 (Low, Irregular, High, Weak, Passive, Yes) label=Yes
   - removed from G (do not cover): {<?, ?, ?, ?, ?, No>; <?, ?, Low, ?, ?, ?>}
   - S generalised <Low, Irregular, Low, ?, Passive, No> -> <Low, Irregular, ?, ?, Passive, ?>
S4 = {<Low, Irregular, ?, ?, Passive, ?>}
G4 = {<?, ?, ?, ?, Passive, ?>; <?, Irregular, ?, ?, ?, ?>; <Low, ?, ?, ?, ?, ?>}

Step 5: E5 (Medium, Irregular, High, Strong, Active, No) label=No
   - kept specialisation <Low, Irregular, ?, ?, ?, ?>
   - rejected <High, Irregular, ?, ?, ?, ?> (not more general than any s in S)
   - rejected <?, Irregular, Low, ?, ?, ?> (not more general than any s in S)
   - rejected <?, Irregular, ?, Weak, ?, ?> (not more general than any s in S)
   - kept specialisation <?, Irregular, ?, ?, Passive, ?>
   - rejected <?, Irregular, ?, ?, ?, Yes> (not more general than any s in S)
   - removed from G (less general than another member): {<?, Irregular, ?, ?, Passive, ?>; <Low, Irregular, ?, ?, ?, ?>}
S5 = {<Low, Irregular, ?, ?, Passive, ?>}
G5 = {<?, ?, ?, ?, Passive, ?>; <Low, ?, ?, ?, ?, ?>}
```

9.4 Console log: ID3 entropy and gain per attribute per node

```
[root] n=14 counts={'Yes': 7, 'No': 7} H=1.0000
     Attendance   gain=0.1636 score=0.1636
     Assignments  gain=0.0611 score=0.0611
     LMS          gain=0.1369 score=0.1369
     PriorCGPA    gain=0.4083 score=0.4083
[root/PriorCGPA=Weak] n=7 counts={'Yes': 6, 'No': 1} H=0.5917
     Attendance   gain=0.3060 score=0.3060
     Assignments  gain=0.1981 score=0.1981
     LMS          gain=0.1981 score=0.1981
[root/PriorCGPA=Weak/Attendance=High] n=2 counts={'Yes': 1, 'No': 1} H=1.0000
     Assignments  gain=1.0000 score=1.0000
     LMS          gain=1.0000 score=1.0000
[root/PriorCGPA=Strong] n=7 counts={'Yes': 1, 'No': 6} H=0.5917
     Attendance   gain=0.3060 score=0.3060
     Assignments  gain=0.1281 score=0.1281
     LMS          gain=0.1981 score=0.1981
[root/PriorCGPA=Strong/Attendance=Low] n=2 counts={'Yes': 1, 'No': 1} H=1.0000
     Assignments  gain=1.0000 score=1.0000
     LMS          gain=1.0000 score=1.0000

Printed tree:
PriorCGPA = Weak:
```

```
    Attendance = Low:
        -> Yes  (n=3, {'Yes': 3})
    Attendance = Medium:
        -> Yes  (n=2, {'Yes': 2})
    Attendance = High:
        Assignments = Irregular:
            -> Yes  (n=1, {'Yes': 1})
        Assignments = Regular:
            -> No  (n=1, {'No': 1})
PriorCGPA = Strong:
    Attendance = Low:
        Assignments = Irregular:
            -> Yes  (n=1, {'Yes': 1})
        Assignments = Regular:
            -> No  (n=1, {'No': 1})
    Attendance = Medium:
        -> No  (n=3, {'No': 3})
    Attendance = High:
        -> No  (n=2, {'No': 2})

Predictions on Test Set T (full tree): T1=Yes, T2=Yes, T3=No, T4=Yes, T5=No, T6=Yes
```

9.5 Test cases

Test	Input / purpose	Expected behaviour	Observed output	Pass/fail
TC1	Candidate Elimination on Dataset A, S/G after every example vs hand trace (Task 4)	S1..S5, G1..G5 identical to hand trace	S5={<Low, Irregular, ?, ?, Passive, ?>} ; G5={<?, ?, ?, ?, Passive, ?>; <Low, ?, ?, ?, ?, ?>}	PASS
TC2	Entropy of pure subset and of 7Y/7N set	0.0000 and 1.0000	0.0000 and 1.0000	PASS
TC3	Root information gains (Attendance, Assignments, LMS, PriorCGPA)	0.1636, 0.0611, 0.1369, 0.4083	0.1636, 0.0611, 0.1369, 0.4083	PASS
TC4	Tree from Dataset B classifies all 14 training rows; 8 rules; depth 3, 13 nodes	14/14 correct; 8 rules	14/14 correct; 8 rules; {'nodes': 13, 'leaves': 8, 'depth': 3}	PASS
TC5	Predictions/accuracy of full tree on Test Set T vs Task 8 hand calculation	Yes,Yes,No,Yes,No,Yes ; 3/6 = 0.5	Yes,Yes,No,Yes,No,Yes ; 3/6 = 0.5000	PASS
TC6	Empty version space (E1-E5+E6 and E1-E5+E7+E8) must be reported, not crash	Collapse reported at E6; at E8	collapse at E6; collapse at E8	PASS
TC7	Unseen value Attendance='Very Low' (policy: stop at current node and return its majority class)	No crash; returns majority class of node PriorCGPA=Weak (Yes)	predict -> Yes	PASS
TC8	Tie on gain at node (PriorCGPA=Weak, Attendance=High): Assignments vs LMS both 1.0000	Deterministic: first attribute in declared order (Assignments), identical on repeated runs	run1 chooses Assignments, run2 (shuffled rows) chooses Assignments; trees identical: True	PASS

Unseen-value policy (TC7): when the value is not a branch of the current node, the tree stops at that node and returns the node's majority class (with the tie rule). This never crashes, and uses only information that the training data support; its weakness (no use of the value's meaning) is addressed in Task 11. **Tie policy (TC8):** the first attribute in the declared order is chosen, and the run on shuffled input rows gave an identical tree.

9.6 Source code (ml_core.py)

```
"""Core ML implementations for ITA06 Assignment 1 (no ML libraries used).
Contains: concept-learning representation, Candidate Elimination, ID3 decision tree."""
import math
from collections import Counter


NULL = "Ø"; ANY = "?"


# ---------------- Datasets ----------------
A_ATTRS = ["Attendance","Assignments","StudyHours","PriorCGPA","LMS","Mentoring"]
A_DOM = {"Attendance":["Low","Medium","High"],"Assignments":["Irregular","Regular"],
         "StudyHours":["Low","High"],"PriorCGPA":["Weak","Strong"],
         "LMS":["Passive","Active"],"Mentoring":["No","Yes"]}
DATA_A = [("E1",("Low","Irregular","Low","Weak","Passive","No"),"Yes"),
          ("E2",("Low","Irregular","Low","Strong","Passive","No"),"Yes"),
          ("E3",("High","Regular","High","Strong","Active","Yes"),"No"),
          ("E4",("Low","Irregular","High","Weak","Passive","Yes"),"Yes"),
          ("E5",("Medium","Irregular","High","Strong","Active","No"),"No")]
B_ATTRS = ["Attendance","Assignments","LMS","PriorCGPA"]
B_DOM = {"Attendance":["Low","Medium","High"],"Assignments":["Irregular","Regular"],
         "LMS":["Passive","Active"],"PriorCGPA":["Weak","Strong"]}
 B = """Low Irregular Passive Weak Yes
Low Irregular Passive Strong Yes
Low Regular Active Weak Yes
Low Regular Active Strong No
Medium Irregular Passive Weak Yes
Medium Irregular Active Strong No
Medium Regular Passive Weak Yes
Medium Regular Active Strong No
High Irregular Passive Weak Yes
High Irregular Active Strong No
High Regular Passive Strong No
High Regular Active Weak No
Medium Irregular Passive Strong No
Low Irregular Active Weak Yes"""
 T = """Low Irregular Active Strong No
Medium Regular Active Weak No
High Regular Passive Weak Yes
Medium Irregular Active Weak Yes
Low Regular Passive Strong No
High Irregular Active Weak Yes"""
def _parse(s, prefix=""):
    out=[]
    for i,l in enumerate(s.splitlines(),1):
        p=l.split(); out.append((f"{prefix}{i}",tuple(p[:4]),p[4]))
    return out
DATA_B =  _parse( B); DATA_T =  _parse( T,"T")
EXTRA_A = [("E6",("Low","Irregular","Low","Weak","Passive","No"),"No"),
           ("E7",("Medium","Regular","High","Weak","Passive","No"),"Yes"),
           ("E8",("Low","Regular","High","Strong","Active","Yes"),"Yes")]
EXTRA_B = [("N1",("Very Low","Irregular","Passive","Weak"),"Yes"),
```

```
          ("N2",("High","Regular","Active","Strong"),"Yes"),
          ("N3",("Low","Regular","Passive","Weak"),"Yes")]

# ---------------- Concept learning ----------------
def covers(h, x):
    """True if hypothesis h classifies instance x as positive."""
    return all(hv == ANY or (hv != NULL and hv == xv) for hv, xv in zip(h, x))

def more_general_or_equal(h1, h2):
    """h1 >= g h2 : every instance covered by h2 is covered by h1."""
    if NULL in h2: return True              # h2 covers nothing
    return all(a == ANY or (a == b and a != NULL) for a, b in zip(h1, h2))

def min_generalize(s, x):
    return tuple(xv if sv == NULL else (sv if sv == xv else ANY) for sv, xv in zip(s, x))

def min_specializations(g, x, dom, attrs):
    out = []
    for i, gv in enumerate(g):
        if gv == ANY:
            for v in dom[attrs[i]]:
                if v != x[i]:
                    out.append(g[:i] + (v,) + g[i+1:])
    return out

def fmt(h): return "<" + ", ".join(h) + ">"
def fmtset(S): return "{" + "; ".join(fmt(h) for h in sorted(S)) + "}" if S else "{}  (EMPTY)"

def candidate_elimination(data, attrs, dom, verbose=True, log=None):
    n = len(attrs)
    S = {tuple([NULL]*n)}; G = {tuple([ANY]*n)}
    trace = [("init", None, None, set(S), set(G), "initial boundaries")]
    if verbose: print(f"S0 = {fmtset(S)}\nG0 = {fmtset(G)}")
    for step, (name, x, y) in enumerate(data, 1):
        notes = []
        if y == "Yes":
            removed = {g for g in G if not covers(g, x)}
            G -= removed
            if removed: notes.append("removed from G (do not cover): " + fmtset(removed))
            newS = set()
            for s in S:
                if covers(s, x): newS.add(s); continue
                h = min_generalize(s, x)
                if any(more_general_or_equal(g, h) for g in G):
                    newS.add(h); notes.append(f"S generalised {fmt(s)} -> {fmt(h)}")
                else: notes.append(f"generalisation {fmt(h)} dropped (no g in G is more general)")
            S = {s for s in newS if not any(s != t and more_general_or_equal(s, t) for t in newS)}
        else:
            removed = {s for s in S if covers(s, x)}
            S -= removed
            if removed: notes.append("removed from S (cover negative): " + fmtset(removed))
            newG = set()
            for g in G:
                if not covers(g, x): newG.add(g); continue
                cands = min_specializations(g, x, dom, attrs)
                for h in cands:
                    if any(more_general_or_equal(h, s) for s in S):
                        newG.add(h); notes.append(f"kept specialisation {fmt(h)}")
                    else: notes.append(f"rejected {fmt(h)} (not more general than any s in S)")
            G = {g for g in newG if not any(g != t and more_general_or_equal(t, g) for t in newG)}
            dropped = newG - G
            if dropped: notes.append("removed from G (less general than another member): " + fmtset(dropped))
        trace.append((step, name, y, set(S), set(G), notes))
        if verbose:
            print(f"\nStep {step}: {name} {x} label={y}")
            for nt in notes: print("   -", nt)
            print(f"S{step} = {fmtset(S)}\nG{step} = {fmtset(G)}")
        if not S or not G:
            if verbose: print(f"*** VERSION SPACE EMPTY after {name}: data inconsistent with conjunctive hypothesis space ***")
            trace.append(("collapse", name, y, set(S), set(G), "version space empty"))
            return S, G, trace, name
    return S, G, trace, None

def version_space(S, G, attrs, dom):
    """Enumerate all conjunctive hypotheses h with some s<=h<=some g."""
    import itertools
    space = [dom[a]+[ANY] for a in attrs]
    return [h for h in itertools.product(*space)
            if any(more_general_or_equal(h, s) for s in S) and any(more_general_or_equal(g, h) for g in G)]

# ---------------- Decision tree (ID3) ----------------
def entropy(labels):
    n = len(labels)
    if n == 0: return 0.0
    return abs(-sum((c/n)*math.log2(c/n) for c in Counter(labels).values()))   # abs avoids -0.0

def majority(labels):
    c = Counter(labels)
    top = max(c.values())
    winners = [k for k, v in c.items() if v == top]
    return "Yes" if "Yes" in winners else sorted(winners)[0]   # tie -> Yes (FN costlier)

def info_gain(rows, i):
    labels = [r[1] for r in rows]; n = len(rows)
    rem = 0.0
    for v in set(r[0][i] for r in rows):
        sub = [r[1] for r in rows if r[0][i] == v]
        rem += len(sub)/n*entropy(sub)
    return entropy(labels) - rem

def split_info(rows, i):
    n = len(rows)
    return -sum((c/n)*math.log2(c/n) for c in Counter(r[0][i] for r in rows).values())

class Node:
    def __init__(s, **k): s.__dict__.update(k)

def id3(rows, attrs_idx, names, doms, depth=0, max_depth=None, min_split=2, min_gain=0.0,
        measure="gain", verbose=False, path="root", forced_root=None, min_leaf=1):
    labels = [r[1] for r in rows]
    maj = majority(labels)
    node = Node(leaf=True, label=maj, n=len(rows), counts=dict(Counter(labels)), attr=None, children={}, depth=depth)
    if len(set(labels)) == 1 or not attrs_idx: return node
    if max_depth is not None and depth >= max_depth: return node
    if len(rows) < min_split: return node
    scores = []
    for i in attrs_idx:
        g = info_gain(rows, i)
        sc = g
        if measure == "gainratio":
            si = split_info(rows, i); sc = g/si if si > 0 else 0.0
        scores.append((i, g, sc))
    if verbose:
        print(f"[{path}] n={len(rows)} counts={dict(Counter(labels))} H={entropy(labels):.4f}")
        for i, g, sc in scores: print(f"     {names[i]:<12} gain={g:.4f} score={sc:.4f}")
    best = max(scores, key=lambda t: (round(t[2], 12), -t[0]))   # tie -> lowest attribute index (deterministic)
    if forced_root is not None and depth == 0:
        best = [t for t in scores if t[0] == forced_root][0]
    if best[1] <= min_gain: return node
    if min_leaf > 1:
        sizes = [sum(1 for r in rows if r[0][best[0]] == v) for v in set(r[0][best[0]] for r in rows)]
        if min(sizes) < min_leaf: return node
    node.leaf = False; node.attr = best[0]; node.attr_name = names[best[0]]; node.gain = best[1]
    rest = [i for i in attrs_idx if i != best[0]]
    for v in doms[names[best[0]]]:
```

```
        sub = [r for r in rows if r[0][best[0]] == v]
        if not sub:
            node.children[v] = Node(leaf=True, label=maj, n=0, counts={}, attr=None, children={}, depth=depth+1)
        else:
            node.children[v] = id3(sub, rest, names, doms, depth+1, max_depth, min_split, min_gain,
                                   measure, verbose, path+f"/{names[best[0]]}={v}", None, min_leaf)
    return node


def predict(node, x):
    while not node.leaf:
        v = x[node.attr]
        if v not in node.children: return node.label     # unseen value -> majority at this node
        node = node.children[v]
    return node.label


def stats(node):
    if node.leaf: return dict(nodes=1, leaves=1, depth=0)
    ch = [stats(c) for c in node.children.values()]
    return dict(nodes=1+sum(c["nodes"] for c in ch), leaves=sum(c["leaves"] for c in ch), depth=1+max(c["depth"] for c in ch))


def rules(node, names, prefix=()):
    if node.leaf:
        cond = " AND ".join(prefix) if prefix else "TRUE"
        return [f"IF {cond} THEN AtRisk = {node.label}   [n={node.n}, {node.counts}]"]
    out = []
    for v, c in node.children.items():
        out += rules(c, names, prefix + (f"{node.attr name} = {v}",))
    return out


def show(node, indent=""):
    if node.leaf: return [f"{indent}-> {node.label}  (n={node.n}, {node.counts})"]
    out = []
    for v, c in node.children.items():
        out.append(f"{indent}{node.attr name} = {v}:")
        out += show(c, indent + "    ")
    return out


def accuracy(node, data):
    return sum(predict(node, x) == y for _, x, y in data)/len(data)


def confusion(node, data):
    tp=fp=tn=fn=0
    for  , x, y in data:
        p = predict(node, x)
        if y=="Yes" and p=="Yes": tp+=1
        elif y=="No" and p=="Yes": fp+=1
        elif y=="No" and p=="No": tn+=1
        else: fn+=1
    return tp, fp, tn, fn


def p yes(node, alpha=1.0):
    y = node.counts.get("Yes", 0)
    return (y + alpha) / (node.n + 2*alpha)


def predict_proba(node, x, unseen="majority"):
    """Return (P(AtRisk=Yes), leaf) using Laplace-smoothed leaf frequencies.
    unseen: 'majority' -> stop at the current node; 'conservative' -> follow the child with highest P(Yes)."""
    while not node.leaf:
        v = x[node.attr]
        if v not in node.children:
            if unseen == "conservative":
                real = [c for c in node.children.values() if c.n > 0]
                node = max(real, key=lambda c: p_yes(c)); continue
            return p_yes(node), node
        node = node.children[v]
    return p_yes(node), node


def predict_thr(node, x, thr=0.5, unseen="majority"):
    return "Yes" if predict_proba(node, x, unseen)[0] >= thr else "No"


def explain(node, x, names, unseen="majority"):
    path = []
    while not node.leaf:
        v = x[node.attr]
        if v not in node.children:
            path.append(f"{node.attr_name} = {v} (value not seen in training)"); break
        path.append(f"{node.attr name} = {v}"); node = node.children[v]
    p, leaf = predict_proba(node, x, unseen) if False else (p_yes(node), node)
    return path, p, node


def metrics(tp, fp, tn, fn):
    acc = (tp+tn)/max(1, tp+fp+tn+fn)
    prec = tp/(tp+fp) if tp+fp else 0.0
    rec = tp/(tp+fn) if tp+fn else 0.0
    f1 = 2*prec*rec/(prec+rec) if prec+rec else 0.0
    return dict(acc=acc, prec=prec, rec=rec, f1=f1, tp=tp, fp=fp, tn=tn, fn=fn)
```

Task 10 – Experimental Analysis, Theory vs Experiment and Recommendation

10A.1 Dataset characteristics

Dataset	Examples	Attributes	Class balance (Yes : No)	Attribute cardinalities	Instance space	Coverage
A	5	6	3 : 2 (60% Yes)	3,2,2,2,2,2	96	5/96 = 5.21%
B	14	4	7 : 7 (50% Yes)	3,2,2,2	24	14/24 = 58.33%
T	6	4	3 : 3 (50% Yes)	3,2,2,2	24	6/24 = 25.00% (none overlaps B)
C (synthetic)	1,000	4	436 : 564 (43.6% Yes)	3,2,2,2	24	24/24 = 100% (about 42 per cell)

Dataset C generator (documented synthetic): attributes drawn independently with P(Attendance = Low/Medium/High) = 0.30/0.40/0.30, P(Irregular) = 0.45, P(Passive) = 0.45, P(Weak) = 0.40; score = 2·[Low] + 1·[Medium] + [Irregular] + [Passive] + [Weak]; AtRisk = Yes if score ≥ 3; a configurable fraction of labels is then flipped (0%, 5%, 10%). Seed 42; 1,000 records, 700 for training and 300 for validation. In the noise experiments the noise is injected into training labels only and accuracy is measured against clean validation labels.

10A.2 Performance on T and cross-validation (AtRisk = Yes positive)

Model (trained on B), tested on T	Accuracy	Precision	Recall	F1	Confusion matrix
M0 depth 0	0.5000	0.5000	1.0000	0.6667	TP 3 / FP 3 / FN 0 / TN 0
M1 depth 1	0.8333	0.7500	1.0000	0.8571	TP 3 / FP 1 / FN 0 / TN 2
Depth 2	0.5000	0.5000	1.0000	0.6667	TP 3 / FP 3 / FN 0 / TN 0
M2 full tree	0.5000	0.5000	0.6667	0.5714	TP 2 / FP 2 / FN 1 / TN 1
min_split=4	0.5000	0.5000	1.0000	0.6667	TP 3 / FP 3 / FN 0 / TN 0

Model, leave-one-out over 20 records (B + T)	Accuracy	Precision	Recall	F1	Confusion matrix
M0 depth 0	0.0000	0.0000	0.0000	0.0000	TP 0 / FP 10 / FN 10 / TN 0
M1 depth 1	0.8500	0.8182	0.9000	0.8571	TP 9 / FP 2 / FN 1 / TN 8
Depth 2	0.8500	0.8182	0.9000	0.8571	TP 9 / FP 2 / FN 1 / TN 8
M2 full tree	0.5500	0.5455	0.6000	0.5714	TP 6 / FP 5 / FN 4 / TN 5
min_split=4	0.7000	0.7500	0.6000	0.6667	TP 6 / FP 2 / FN 4 / TN 8

Note on LOO for M0: with 10 Yes and 10 No records, leaving out one record makes the *other* class the majority, so the majority predictor is wrong every time (0.00 accuracy); this is a known artefact of LOO with balanced classes, not a bug. The full tree reaches recall 0.60 and F1 0.57 in LOO; the depth-1 tree achieves recall 0.90 (9 of 10 at-risk students found) and accuracy 0.85. The depth-1 tree is the only model near the 90% recall requirement; recall on the six-student T is 1.00 for it but one FP, so precision is 0.75.

10A.3 Effect of training-set size

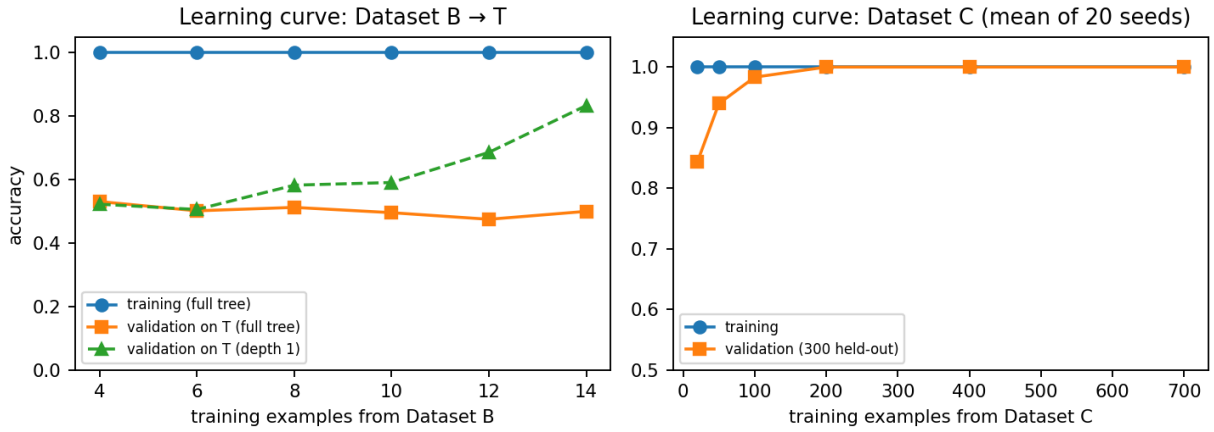


Figure 6 – Learning curves (left: Dataset B → T, 200 random subsets per size; right: Dataset C).

Training size (B)	Training accuracy (full tree)	Validation on T (full tree)	Validation on T (depth 1)
4	1.0000	0.5308	0.5225
6	1.0000	0.5017	0.5058
8	1.0000	0.5125	0.5825
10	1.0000	0.4958	0.5908
12	1.0000	0.4750	0.6858
14	1.0000	0.5000	0.8333

Training size (C)	Training accuracy	Validation accuracy (300 held out)
20	1.0000	0.8435
50	1.0000	0.9397
100	1.0000	0.9833
200	1.0000	1.0000
400	1.0000	1.0000
700	1.0000	1.0000

On B, the full tree has perfect training accuracy at every size (it can always memorise) while validation on T stays near 0.50 and does not improve with more data (0.53 at 4 examples, 0.50 at 14), a sign that variance, not shortage of examples alone, dominates. The depth-1 tree improves as data increase (0.52 → 0.83). On Dataset C the validation accuracy rises from 0.84 (20 examples) to 1.00 (200 examples or more): with 24 cells and a clean rule, about 8 examples per cell are enough for the tree to recover the rule exactly.

10A.4 Effect of model complexity

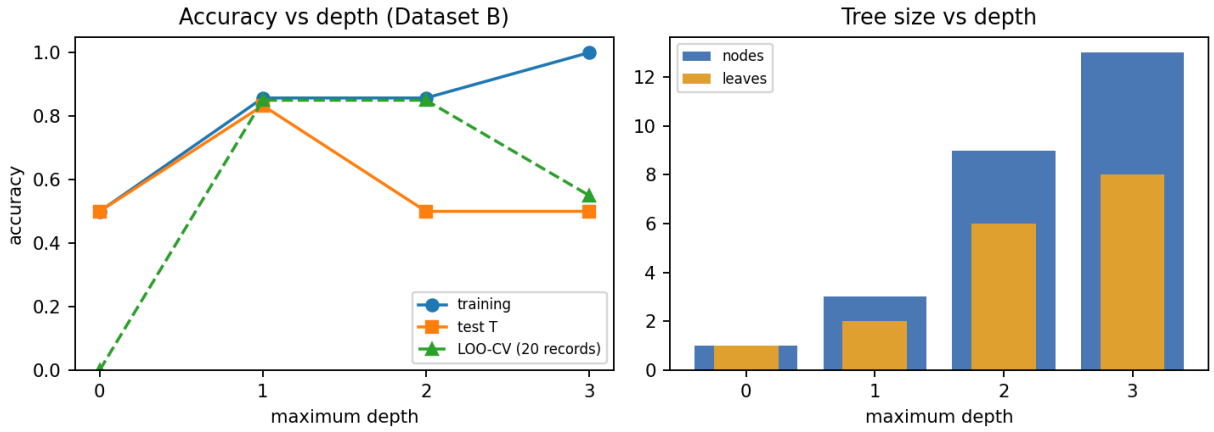


Figure 7 – Accuracy and tree size versus maximum depth (Dataset B).

Max depth	Nodes	Leaves	Training acc.	Test acc. (T)	LOO accuracy
0	1	1	0.5000	0.5000	0.0000
1	3	2	0.8571	0.8333	0.8500
2	9	6	0.8571	0.5000	0.8500
3 (full)	13	8	1.0000	0.5000	0.5500

Training accuracy rises monotonically with depth (0.50 → 0.86 → 0.86 → 1.00) while test and LOO accuracy peak at depth 1 and then fall (0.83/0.85 → 0.50/0.55): the classic under-fit / good-fit / over-fit pattern. Tree size grows from 1 node to 13.

10A.5 Effect of label noise (Dataset C, mean of 20 runs)

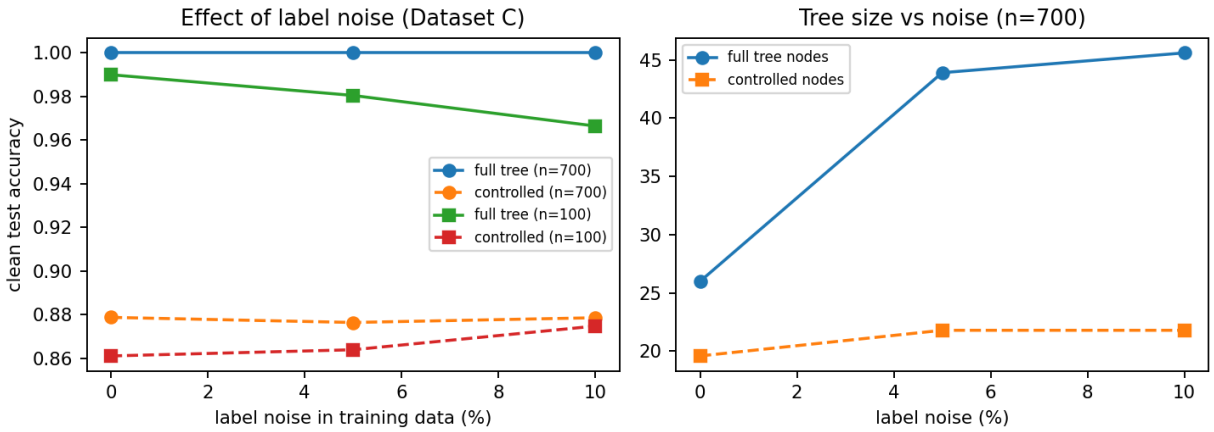


Figure 8 – Effect of noise on accuracy and tree size.

Training size	Noise	Full tree: clean accuracy	Full tree: nodes / leaves	Controlled: clean accuracy	Controlled: nodes / leaves
100	0%	0.9900	24.7 / 13.4	0.8612	18.7 / 10.4
100	5%	0.9805	29.9 / 16.1	0.8640	21.9 / 12.1
100	10%	0.9665	33.4 / 17.9	0.8748	24.1 / 13.2
700	0%	1.0000	26.0 / 14.0	0.8788	19.6 / 10.8
700	5%	1.0000	43.9 / 22.9	0.8765	21.8 / 11.9
700	10%	1.0000	45.6 / 23.8	0.8787	21.8 / 11.9

The controlled tree uses min_samples_split = max(8, n/10) and min_gain = 0.01. **Findings.** Noise barely lowers the accuracy of the full tree on 700 records (1.000 → 1.000), because each of the 24 cells holds about 29 training records so the majority label wins, but it nearly doubles its size (26 → 45.6 nodes, 14 → 23.8 leaves) as the tree

splits further to fit flipped labels, and its training accuracy falls to 0.90 (the noise rate). With only 100 records the full tree does lose accuracy (0.990 → 0.967) while growing from 24.7 to 33.4 nodes. The controlled tree is stable in size (about 20–22 nodes at 700), but its fixed pre-pruning setting costs about 12 points of accuracy even without noise (0.879 versus 1.000): over-pruning hides the real rule. The best setting is therefore data dependent, not a fixed pruning strength.

10A.6 Candidate Elimination versus decision tree

Running Candidate Elimination on Dataset B (same four attributes): S_1 = ⟨Low, Irregular, Passive, Weak⟩, S_2 = ⟨Low, Irregular, Passive, ?⟩, S_3 = ⟨Low, ?, ?, ?⟩ (rows 1–3, all Yes), then **row 4 (Low, Regular, Active, Strong → No) is covered by S_3**: S is removed and no specialisation of G can stay above an empty S, so **S and G are both empty and the version space collapses at example 4** (28.6% through the data). On Dataset C (500 records) the collapse is at record 6 for 0%, 5% and 10% noise, so it happens even without noise, because the AtRisk rule is not a conjunction.

Aspect	Candidate Elimination (conjunctions)	Decision tree (ID3)
Can represent	Single conjunctions only (e.g. Low AND Passive); no disjunction	Any Boolean function of the attributes: DNF rule sets (8 rules in Task 6)
Result on Dataset B	Version space empty after example 4	Consistent tree (14/14) with 13 nodes
Noise / inconsistent labels	Collapses with a single contradiction (E6 vs E1)	Degrades gradually; can prune and use majority leaves
Unseen cases	Votes of the version space; no answer after collapse	Always gives an answer (leaf or node majority)
Data needed	Few, but must be noise free and conjunctive	More data, tolerant of noise
Interpretability	Very high when it works	High (path = reason)

10B. Theory versus experimental behaviour

- **Predicted overfitting of the full tree – supported.** Theory predicted a 100%-training tree to generalise poorly; experiment: 14/14 training but 3/6 on T, LOO accuracy 0.55, and the gap grows with depth.
- **Predicted collapse of the version space – supported, and stronger than expected.** Task 4/5 predicted that non-conjunctive data would break CE; it collapsed at example 4 of Dataset B and at record 6 of Dataset C even with no noise.
- **Dataset characteristics.** Small data and low coverage (5% for A, 58% for B) lead to unstable trees; with 24 cells and 700 records Dataset C removes the problem (accuracy 1.00 from 200 examples); noise mainly inflates tree size when each cell has many records.
- **Inductive bias.** The conjunctive bias is too strong for this data (fails at example 4); the tree's weaker bias fits the data but needs a preference for simplicity (depth control).
- **Heuristic selection.** Information gain and gain ratio chose the same root and gave 13-node trees; they differ only on small subsets, so the choice mattered less than depth control. Gain ratio rejected the StudentID attribute (0.2626 versus 1.0000 for gain).
- **Model complexity.** Training accuracy rose with depth and test accuracy peaked at depth 1 – exactly the bias–variance picture.
- **Limitations / threats to validity.** T has 6 records and all are unseen cells; LOO on 20 records is high variance; Dataset C is synthetic with a simple rule and independent attributes; the controlled setting was not tuned per dataset; results for noise are for random flips, not for systematic advisor bias.

Evidence-based recommendation

Recommendation: use decision-tree learning (a controlled, pruned tree) for the early-warning system; keep Candidate Elimination only as a diagnostic tool.

Criterion	Candidate Elimination	Full tree	Controlled (depth 1) tree
Accuracy (T / LOO)	No hypothesis after example 4	0.50 / 0.55	0.83 / 0.85
Recall on AtRisk (T / LOO)	–	0.67 / 0.60	1.00 / 0.90

Criterion	Candidate Elimination	Full tree	Controlled (depth 1) tree
Precision (T / LOO)	–	0.50 / 0.55	0.75 / 0.82
Complexity	Boundaries of 1–5 hypotheses	13 nodes, 8 leaves	3 nodes, 2 leaves
Interpretability	High, until collapse	High but includes single-student rules	Very high: one rule on PriorCGPA
Noise handling	None	Memorises noise	Majority leaves
Unseen cases	Votes only inside a version space	Node majority	Node majority
Data requirement	Noise-free, conjunctive	Larger sample needed	Works with 14–20 records

The decision is based on measured evidence: CE collapses on the very first contradictory record, whereas the depth-1 tree achieves recall 0.90 and accuracy 0.85 in leave-one-out and 5/6 on T, while the full tree is no better than chance on T. **Conditions where Candidate Elimination would be preferable:** when the data are known to be noise free and the concept is truly a single conjunction (for example a fixed regulation such as “Attendance < 65% AND Assignments Irregular ⇒ detained”), when only a handful of labelled cases exist and the version space is needed to choose the next case to label, or when auditing whether a proposed rule is consistent with the data. The depth-1 tree still misses 10% of at-risk students in LOO, so it is not sufficient for the 90% recall rule the university introduces later; Task 11 addresses that.

Task 11 – Day-4 Design Challenge: Re-evaluate and Adapt the Design

A. Impact analysis

Change	Original assumption violated	Affects	Consequence for concept learning	Consequence for decision trees
≈2,000 records, weekly arrivals	Small, fixed dataset	Data	More examples reveal non-conjunctive structure; CE collapses sooner	Deeper trees are justified, but retraining must be repeated and tracked
5–10% inconsistent labels	Noise-free labels	Data, learning algorithm	One contradiction empties the version space (E6)	Needs majority leaves and pruning; otherwise it memorises noise (N2)
New values (Very Low) and combinations	Fixed attribute values	Hypothesis space	Domain of ? and of specialisation is incomplete; unseen value has no rule	A split has no branch for the value; policy needed
Recall ≥ 90%, explanation for every flag	Accuracy-only evaluation	Evaluation	Consistent hypothesis gives no recall control	Add decision threshold and class weighting; keep rule explanations
Real target concept is a disjunction	Conjunctive target concept	Hypothesis space	Cannot express "Passive OR Low attendance" (E3/E7/E8)	Trees express disjunctions as several leaves

N2 versus E6. E6 is an *exact duplicate* of E1's attribute values with the opposite label, so no hypothesis can be consistent with both; a method assuming consistent data (CE) must declare failure and the version space is empty. N2 = ⟨High, Regular, Active, Strong⟩ → Yes is a *new combination* that contradicts the learned pattern (High attendance and Strong CGPA is safe), not an identical record. A noise-tolerant method treats both as probable label noise (the majority of similar records wins, a leaf with n ≥ the minimum size is kept) and keeps a working model; a method that assumes consistent data either collapses on E6 or, like ID3, grows an extra LMS split to fit N2 exactly.

B. Re-evaluation of the original design

Candidate Elimination with the extra records

Added example	Attributes	S after	G after	What happens
E6 (–)	Low,Irr,Low,Weak,Passive,No	{ } (empty)	{ } (empty)	Collapse: E6 has the same attributes as E1 (positive); S_5 covers E6 so S is emptied; no specialisation of G stays above an empty S.
E7 (+)	Medium,Reg,High,Weak,Passive,No	{(?,?,?,?,Passive,?)}	{(?,?,?,?,Passive,?)}	G member ⟨Low,?,…⟩ is removed (does not cover E7); S generalised to LMS = Passive. VS has one hypothesis.
E8 (+) after E7	Low,Reg,High,Strong,Active,Yes	{ } (empty)	{ } (empty)	E8 has LMS = Active, so the only G member ⟨?,…,Passive,?⟩ is removed; G is empty, so the version space collapses.

The data now contain two different groups of at-risk students (E1/E2/E4 with Low attendance and Passive LMS; E7 and E8 with Medium/Low attendance and Active or Passive LMS), i.e. a *disjunctive* concept that no single conjunction can represent. The original CE design fails.

Decision tree rebuilt with N1–N3

Tree	Nodes	Leaves	Depth	Training accuracy
Original (14 examples)	13	8	3	14/14
Rebuilt with N1–N3 (17 examples)	19	12	4	17/17

Changes: N1 adds an Attendance = Very Low branch under PriorCGPA = Weak (1 student, Yes) and, because Very Low is now in the Attendance domain, an *empty* branch under PriorCGPA = Strong (no examples, so the parent majority No is used, a risky default for a student with the lowest attendance); N3 enlarges the (Weak, Low) leaf to 4 Yes; **N2** makes the (Strong, High) leaf impure, so the tree adds an Assignments split and then an LMS split under Regular – Passive → No (row 11) and Active → Yes (N2) – which costs 4 extra nodes, 2 extra leaves and one extra level just to fit one suspicious record (N1 accounts for the other 2 nodes and 2 leaves). **Handling of N1 in the original tree:** the value Very Low is not a branch of the Attendance node under PriorCGPA = Weak, so the tree stops there and returns that node's majority class (Yes), which is correct here by luck (N1 and N3 correct, N2 wrong: the original tree predicts No for N2).

90% recall and explanations. On the old data the full tree has recall 0.60 in LOO and the depth-1 tree 0.90. On a simulated 2,000-record scale-up (below), the full tree has recall 0.686 on audited test records, so the **90% requirement is not met** by the original tree, and it misses all 22 Very Low test students (recall 0.00 on them) because unseen values default to the node majority. Both trees do produce IF–THEN explanations (path from root to leaf), so the *explanation* requirement is met; CE can neither meet recall (no hypothesis) nor explain once the version space is empty. Verdict: CE – not suitable; original tree – explanation OK, recall and unseen values not OK.

C. Adaptation or redesign

Decision: MODIFY. The decision-tree family is retained (it expresses disjunctions, gives path explanations and tolerates noise), Candidate Elimination is retired from the decision path, and the induction, unseen-value handling, evaluation and explanation are changed.

Area	Original design	Revised design
Hypothesis representation	Conjunction (CE) or fully grown ID3 tree	Pruned tree = disjunction (DNF) of conjunctive rules; leaves store counts and Laplace-smoothed P(Yes) = (Yes + 1)/(n + 2)
Induction	ID3, information gain, grow until pure	Gain ratio (cardinality increases), pre-pruning (min_samples_split, min_gain) then post-pruning on an audited validation set; retrain weekly
Noise	None	Majority-label leaves; audit and relabel inconsistent records; do not split to isolate one record
Unseen values	Stop at node, use majority	Ordinal back-off: map Very Low to the most at-risk ordered neighbour (Low) or follow the child with highest P(Yes), mark the flag "reviewed by advisor", retrain when ≥ 30 records exist
Recall requirement	Accuracy, tie → Yes	Flag if P(Yes) ≥ θ; θ chosen on an audited validation set to reach ≥ 92% recall (safety margin over 90%)
Explanation	Rule path	Natural-language sentence: "Flagged because PriorCGPA is Weak and Attendance is Low; 3 of 3 similar students in the history were at risk."

```
train_weekly(records):
   clean  = drop/relabel records flagged by label audit
   tree   = ID3(clean, measure=gain_ratio, min_samples_split=100, min_gain=0.02)   # pre-pruning
   tree   = reduced_error_prune(tree, audited_validation_set)
   theta  = largest threshold in [0.5 .. 0.1] with recall(audited_validation) >= 0.92
   return tree, theta

predict(student):
   p, leaf = descend(tree, student, unseen='follow child with highest P(Yes)')
   flag = (p >= theta)
   explanation = path_to_leaf(student) + counts(leaf)
   if value_unseen: mark_for_advisor_review()
```

Effect on recall and precision (simulation of the new setting). I generated 1,500 training records with a probabilistic rule, 8% flipped labels, no Very Low value; 600 audited test records with 12% Very Low; and a separate 600-record audited validation set (seed 9) to choose θ. The controlled tree (min_samples_split = 100, min_gain = 0.02) has 32 nodes against 46 for the full tree.

Model	Accuracy	Precision	Recall	Share flagged	Recall on Very Low
Original full tree (46 nodes), node-majority fallback	0.780	0.782	0.686	–	0.00
Controlled tree (32 nodes), θ = 0.5	0.767	0.789	0.632	–	0.00
Controlled tree + threshold θ = 0.5 (+ back-off)	0.798	0.807	0.705	38.0%	–
Controlled tree + threshold θ = 0.4 (+ back-off)	0.787	0.737	0.793	46.8%	–
Controlled tree + threshold θ = 0.3 (+ back-off)	0.767	0.692	0.835	52.5%	–
Controlled tree + threshold θ = 0.25 (+ back-off)	0.695	0.602	0.881	63.7%	–
Controlled tree + threshold θ = 0.2 (+ back-off)	0.582	0.510	0.958	81.7%	1.00
Controlled tree + threshold θ = 0.15 (+ back-off)	0.435	0.435	1.000	100.0%	–

Rows with a threshold use the back-off for unseen values, which is why the θ = 0.5 row differs slightly from the majority-fallback row above (the difference is the 22 Very Low students).

θ = 0.2 was selected on the audited validation set (validation recall 0.948); on the test set it gives recall **0.958** (≥ 0.90) at precision 0.510. θ = 0.25 would give only 0.881, which shows the need for a margin. **Cost:** precision falls

from 0.79 to 0.51 and 82% of students are flagged, because in this simulation labels overlap even for the same attribute values. If the university cannot support that many counselling slots, richer attributes (StudyHours, Mentoring) or tiered risk bands (high / medium) are needed.

Unseen values. The original tree misses all 22 Very Low test students, the revised tree catches all 22 (recall 1.00 on this subgroup) by back-off, at the price of flagging them for advisor review. **Expected complexity effect:** 46 → 32 nodes (about 30% smaller) and rules of at most 4 conditions.

D. Evidence-based defence

- **Learning theory and inductive bias.** The conjunctive bias of CE is wrong for a disjunctive concept (E7/E8 vs E1–E5, Task 5 power-set argument); trees have a weaker restriction bias, and the preference for short trees is imposed through pruning.
- **Version-space behaviour.** CE collapsed at E6 (noise), at E8 (disjunction), at example 4 of Dataset B and at record 6 of Dataset C; no CE variant without tolerance to errors can serve a 5–10% noisy label source.
- **Heuristic search.** Greedy gain-based selection is efficient (14 gain evaluations for Dataset B) but unstable on tiny subsets (ties at gain 1.0000); with 2,000 records the subsets are large, and gain ratio guards against many-valued attributes such as Attendance with Very Low.
- **Overfitting.** The full tree overfits (3/6 on T, 0.55 LOO); N2 shows it adding 6 nodes to fit one record; pruning and minimum sizes control this.
- **Experiments.** Noise nearly doubles the full tree (26 → 46 nodes) with no accuracy benefit; the threshold raises recall from 0.63 to 0.96 on audited test records; Very Low recall 0.00 → 1.00.
- **New 90% recall requirement.** Met (0.958 on the audited test set) only by combining a lowered threshold with an audited validation set; accuracy-only training does not meet it (0.63–0.69).

Remaining limitations and monitoring. (1) Precision of about 0.51 means roughly half of the flags are false alarms and flagging 82% may exceed counselling capacity; monitor weekly flag rate, precision (from end-of-semester labels) and advisor feedback on flags. (2) The threshold and the simulated recall depend on the audited labels being representative; monitor recall on a monthly audited sample and re-tune θ. (3) Drift: new attribute values and changes in study mode; monitor the fraction of records with unseen values and trigger retraining/review above 5%. (4) Fairness: report recall and false-positive rates by student group each term. (5) The simulation uses synthetic data; the numbers must be re-measured on the real 2,000 records before deployment.